PADS Pipeline Guide

A walkthrough of the new `pads/` package and the tooling that surrounds it.

This document is aimed at a single concrete question: "I migrated my old pipeline at `code/code_v2.py` into the new `pads/` package — there are a lot of new files (pyproject.toml, tests/, MLflow tracking, pydantic config, …). What are they for, when do I use them, and which ones could I drop?"

The answer is organised as follows:

---

0. Setting up the environment with `uv`

The repo is set up to use `uv`, Astral's modern Python package manager (written in Rust, 10–100× faster than `pip`). It replaces the manual `python -m venv` + `pip install -r requirements.txt` dance with a single command.

0.1 One-shot setup

From a fresh clone, pick the script for your OS:

```
.\scripts\setup.ps1     # Windows (PowerShell)
./scripts/setup.sh      # Linux / WSL2
```

Each script will:

1. Install `uv` (to `%USERPROFILE%\.local\bin` on Windows, `$HOME/.local/bin` on Linux) if it isn't already there. 2. Run `uv sync --all-extras`, which:

- creates the virtualenv (Python 3.10+), - installs `pads` in editable mode (i.e. live source — edit and re-run, no reinstall), - installs every runtime dependency from `pyproject.toml`, - installs the optional `dev` extra (tests, docs tooling), - generates `uv.lock` with the exact resolved versions for reproducibility.

3. Copy `.env.example` → `.env` if you don't have one yet.

Per-OS virtualenvs. Windows uses uv's default `.venv/`; Linux/WSL uses `.venv-linux/`. This keeps the two from clobbering each other when the same checkout is shared (e.g. a `/mnt/c` mount accessed from both Windows and WSL). To make this transparent, `setup.sh` appends `export UV_PROJECT_ENVIRONMENT=.venv-linux` to your `~/.bashrc`, so every subsequent `uv run` in a new shell targets the Linux venv automatically. On Windows nothing extra is needed.

After it finishes, you are ready to go.

0.2 Running commands

There are two equivalent ways:

```
# A) Let uv handle activation for each command (no need to remember Activate.ps1):
uv run pads --data_filename synthetic_dataset.csv --mode prepare_data
uv run python -m pads.cli --data_filename ... --mode all
```

```
uv run pytest -m "not slow"

# B) Activate the venv classically and forget about uv until the deps change:
.\.venv\Scripts\Activate.ps1
pads --data_filename ... --mode all
pytest -m "not slow"
```

0.3 Day-to-day operations

What you want	Command
Re-sync after editing `pyproject.toml`	`uv sync --all-extras`
Add a new dependency	`uv add <pkg>` (writes to `pyproject.toml`)
Add a dev-only dependency	`uv add --dev <pkg>`
Remove a dependency	`uv remove <pkg>`
Upgrade everything to latest compatible	`uv sync --upgrade`
Upgrade one package	`uv lock --upgrade-package <pkg>`
Show the dependency tree	`uv tree`

0.4 What about the old `venv/`?

If you set the project up before `uv`, you probably have an old `venv/` next to the new `.venv/`. Once you have validated that `.venv/` works (the `uv sync` succeeded and `uv run pads --help` prints the CLI help), the old `venv/` is safe to delete:

```
Remove-Item -Recurse -Force venv
```

0.5 Why `uv.lock` matters

`pyproject.toml` says *what* you need (`tensorflow>=2.15`), but doesn't say *which exact build*. `uv.lock` resolves the entire graph once and records the precise versions, hashes and platforms. Commit it to git: anybody who clones the repo and runs `uv sync` gets bit-for-bit the same environment you did. (This is the equivalent of `package-lock.json` in Node or `Cargo.lock` in Rust.)

If you only ever run on your own machine, you can ignore the lockfile — `uv sync` will still work either way.

---

1. Repository at a glance

PADS is an installable Python package whose five steps you run by hand through a small CLI, with optional MLflow tracking:

```
PADS/
■■■ pads/                        # the installable package
■   ■■■ cli.py                   # python -m pads.cli ... (or `pads ...`)
■   ■■■ config.py                # PADSConfig (pydantic)
■   ■■■ pipeline.py              # PADSPipeline: 5 public steps
■   ■■■ tracking.py              # MLflow wrapper (no-op if disabled)
■   ■■■ data/                    # schema, loader, preprocess, windowing, normalizer
■   ■■■ models/                  # layers, mortality LSTM, discharge LSTM, registry
■   ■■■ training/                # train/val/test splits, trainer
■   ■■■ eval/                    # metrics, thresholds, error categorisation
■   ■■■ viz/                     # ROC, error bar+heatmap
■■■ tests/                       # 28 tests (24 unit + 4 end-to-end)
■■■ scripts/
■   ■■■ setup.ps1 / setup.sh     # one-shot environment setup (uv + sync)
■   ■■■ sweep.ps1 / sweep.sh     # run the pipeline over all retrain_types
■   ■■■ build_pdf.py             # rebuild docs/PIPELINE_GUIDE.pdf
```

```
■■■ docs/
■   ■■■ PIPELINE_GUIDE.md       # this file
■   ■■■ PIPELINE_GUIDE.pdf      # same content, as PDF
■■■ notebooks/                  # exploratory Jupyter notebooks
■■■ data/                       # raw input (data/processed/ holds intermediates)
■■■ models/, normalizers/, results/          # weights, scalers, outputs
■■■ pyproject.toml              # package + tooling config
■■■ uv.lock                     # exact resolved versions (commit to git)
```

The pipeline has four operations, exposed via the Python API and the `pads` CLI:

Mode	What it does
`prepare_data`	Train/test split, median imputation values, fitted MinMaxScalers, windowed `.pkl` datasets, and dataset provenance.
`retrain_models`	Retrains the mortality and discharge LSTMs. `--retrain_type` controls freezing strategy.
`calculate_metrics`	Picks optimal thresholds, writes test ROC and threshold JSON.
`inference`	End-to-end prediction + error categorisation + plots.

---

2. The new `pads/` package

The package is split into small, single-purpose modules. Each is independently testable and importable.

2.1 `pads/config.py`

Defines `PADSConfig`, a pydantic model that owns every tunable parameter (paths, retrain strategy, learning rates, seed, batch size, …) and exposes path helpers like `data_dir`, `model_dir`, `results_dir`. It also derives the retrained model filenames from `retrain_type` if you don't override them.

Instead of constants at the bottom of a script, you instantiate `PADSConfig(...)` with the values you want, or pass them via the CLI flags.

2.2 `pads/pipeline.py`

Defines `PADSPipeline(config)` — a thin orchestrator that composes the data, model, training, eval and viz modules into the four public steps:

```
pipeline.prepare_data(data_filename)
pipeline.retrain_models()
pipeline.calculate_metrics()
pipeline.run_inference(data_filename, test_type="full")
```

Each step reads from `data/`, `models/`, `normalizers/` and writes its own artifacts (see Section 12). The constructor seeds Python/NumPy/TF and configures GPU memory growth.

2.3 `pads/data/`

- `schema.py` — declarative dataset schema: `MANDATORY_COLUMNS`, `FEATURES`, `IMPUTE_FIRST_ROW`, `NAN_RULES`, `OUTLIER_CORRECTION`, plus `validate_dataset()` which raises `DatasetValidationError` on any deviation.
- `loader.py` — CSV / JSON / pickle / `stays.txt` IO.
- `preprocess.py` — outlier clipping → first-row imputation → per-column NaN rules.

- `windowing.py` — rolling 48-hour windows for the mortality model, plus the 5-anchor scheme for the discharge model.
- `normalizer.py` — `MinMaxScaler` fit / save / load / transform.

2.4 `pads/models/`

- `layers.py` — custom `SoftmaxTemperature` Keras layer, exposed via `CUSTOM_OBJECTS` for `tf.keras.models.load_model`.
- `mortality.py`, `discharge.py` — `build_*` + `compile_*` for each LSTM.
- `registry.py` — `load_model`, `save_model`, and `load_or_create_model(retrain_type, …)` which implements the four freezing strategies.

2.5 `pads/training/`

- `splits.py` — stay-level train/val/test split helpers and `(X, y)` builders for both rolling and discharge datasets, plus one-hot label encoding.
- `trainer.py` — `class_weights`, `fit`, `set_global_seed`, `configure_gpu`. Callbacks include `EarlyStopping`, `ModelCheckpoint(best.keras)` and `TensorBoard`.

2.6 `pads/eval/`

- `metrics.py` — `BinaryMetrics` dataclass plus `evaluate(y_true, y_pred, threshold)`.
- `thresholds.py` — three threshold selection criteria: `youden`, `min_distance`, `precision_recall`.
- `errors.py` — combines binary mortality and discharge predictions into four clinical categories (`EXITUS <48h`, `EXITUS >48h`, `ALIVE <48h`, `ALIVE >48h`) and assigns a 0..3 severity score to each (real_group, predicted_group) pair.

2.7 `pads/viz/`

- `plots.py` — `plot_roc_combined` (mortality + discharge ROC side by side, with the chosen threshold marked) and `plot_error` (per-stay scatter + bar + heatmap of error severity).

2.8 `pads/cli.py`

The command-line entry point. After `pip install -e .` (driven by `pyproject.toml`), it is also available as a `pads` console script via `[project.scripts]`.

2.9 `pads/tracking.py`

The MLflow wrapper. Detailed in Section 5.

---

3. `pyproject.toml`

`pyproject.toml` is the modern, PEP 621 way to describe a Python package. It replaces the old `setup.py` / `setup.cfg` / `requirements.txt` triad and also stores tool configuration (`mypy`, `pytest`) in one place.

The current `pyproject.toml` declares:

3.1 Project metadata

```
[project]
name = "pads"
version = "0.3.0"
description = "PADS — ICU mortality and discharge prediction pipeline (LSTM)."
```

```
readme = "README.md"
requires-python = ">=3.10"
```

3.2 Runtime dependencies

```
dependencies = [
    "numpy>=1.24", "pandas>=2.0", "pyarrow>=14", "scikit-learn>=1.3", "matplotlib>=3.7",
    "joblib>=1.3", "tqdm>=4.65", "tensorflow>=2.15", "pydantic>=2.5", "tensorboard>=2.20.0",
    "mlflow>=2.10",
]
```

These are the libraries needed for a plain end-to-end run. `pyarrow` is the parquet engine for `pandas.read_parquet` (datasets may be CSV or Parquet). `mlflow` is a core dependency so `uv run pads ...` works with tracking out of the box — but tracking stays **opt-in at runtime**: every call is a no-op unless `MLFLOW_TRACKING_URI` is set.

3.3 Optional dependency groups

```
[project.optional-dependencies]
dev = ["pytest>=7.4", "pytest-cov>=4.1", "mypy>=1.8", "reportlab>=4.5.1", "markdown>=3.10.2"]
```

`dev` carries the test runner and the docs/PDF tooling. If you used `scripts/setup.*` (or ran `uv sync --all-extras`), it is already installed.

For colleagues who prefer plain `pip` over `uv`:

```
pip install -e ".[dev]"
```

Both paths write/read the same `pyproject.toml`, so the two workflows coexist.

3.4 Console script

```
[project.scripts]
pads = "pads.cli:main"
```

After `pip install -e .`, `pads --data_filename …` works from any directory.

3.5 Package discovery

```
[tool.setuptools.packages.find]
include = ["pads*"]
exclude = ["env*", "venv*", "notebooks*", "tests*", "code*"]
```

This is what tells `pip install -e .` to ship only the `pads/` directory and **not** the legacy `code/`, your virtualenvs, the notebooks, or the tests.

3.6 Tooling configuration

`pyproject.toml` also configures two tools:

- `[tool.pytest.ini_options]` — pytest discovers tests under `tests/`. Two custom markers are registered: `slow` (skip with `-m "not slow"`) and `e2e` (the end-to-end pipeline tests). `DeprecationWarning` and `FutureWarning` are silenced for readability.
- `[tool.mypy]` — optional static typing. `ignore_missing_imports = true` accommodates the fact that TensorFlow's stubs are imperfect.

---

4. Running the pipeline by hand + the `sweep` helper

PADS has no workflow engine. You run the four steps yourself with the `pads` CLI, in order. The dependencies between steps are linear, so the mental model is simply "run them top to bottom":

```
prepare_data
        ■  data/processed/{medians_48h.json, train/test_stays.txt,
        ■  lstm_last_48h_*.pkl, icu_expire_flag_*.pkl,
        ■  lstm_disch_3point_48h_*.pkl, outcome_disch_3point_48h_*.pkl,
        ■  source_dataset.json}, normalizers/*.pkl
        ▼
retrain_models                  ■■■ models/lstm_*.keras (base models)
        ■  RETRAINED_<type>_lstm_*.keras
        ▼
calculate_metrics
        ■  data/processed/model_parameters_test.json, results/<rt>/images/roc_combined.png
        ▼
inference
        ■  results/<rt>/{final_result.csv, model_parameters_inference.json},
        ■  results/<rt>/images/{roc_combined_<test_type>,barplot_error,heatmap_error}.png
```

Generated files are split by purpose: raw input lives in `data/`, intermediate training artifacts in `data/processed/`, and final deliverables (predictions + plots) in `results/<retrain_type>/` — one subfolder per model so a sweep over retrain types doesn't overwrite earlier variants.

4.1 One step at a time, or all at once

```
# the whole pipeline in one process (in-process MLflow nesting works automatically)
uv run pads --data_filename synthetic_dataset.csv --mode all

# or step by step — handy for debugging or re-running just the tail
uv run pads --data_filename synthetic_dataset.csv --mode prepare_data
uv run pads --data_filename synthetic_dataset.csv --mode retrain_models --retrain_type full --epochs 1000
uv run pads --data_filename synthetic_dataset.csv --mode calculate_metrics
uv run pads --data_filename synthetic_dataset.csv --mode inference --test_type full
```

Every run parameter is a CLI flag — see Section 11 for the full list. Whether you are on a laptop or a GPU server is determined by **environment variables** (chiefly `MLFLOW_TRACKING_URI`), not by a config file. Drop `--epochs 5` for a smoke test and bump it back up for the real run.

Note there is no built-in skip-if-up-to-date logic (the convenience a `make`/Snakemake DAG would give you): if you re-run `prepare_data`, it regenerates. When running steps separately, `retrain_models` writes a `.mlflow_parent_run_id` file that `calculate_metrics` and `inference` read, so the three nest under one MLflow parent run — run them in that order to keep the grouping.

4.2 Sweeping retrain types

A single run uses exactly one `retrain_type` (default `full`). To compare the four freezing strategies (`full / dense / lstm / scratch`) on the same dataset, use `scripts/sweep.ps1` (Windows) / `scripts/sweep.sh` (Linux). The data-prep step (`prepare_data`) doesn't depend on `retrain_type`, so the script runs it once and then loops `retrain_models → calculate_metrics → inference` per type. Each type writes its own `RETRAINED_<type>_lstm*.keras`, so the outputs don't collide.

```
# Default: sweeps full,dense,lstm,scratch
.\scripts\sweep.ps1 -Dataset synthetic_dataset.csv

# Custom subset / quick smoke run
.\scripts\sweep.ps1 -Dataset synthetic_dataset.csv -Types full,scratch -Epochs 2

# Reuse already-generated data/*.pkl (skip the prep steps)
.\scripts\sweep.ps1 -Dataset synthetic_dataset.csv -SkipData

# With MLflow on, all runs appear in the experiment with `retrain_type` as a param
$env:MLFLOW_TRACKING_URI = "http://localhost:5000"
.\scripts\sweep.ps1 -Dataset synthetic_dataset.csv

# Linux equivalents
./scripts/sweep.sh --dataset synthetic_dataset.csv
```

```
./scripts/sweep.sh --dataset synthetic_dataset.csv --types full,scratch --epochs 2
./scripts/sweep.sh --dataset synthetic_dataset.csv --skip-data
```

The recommended workflow for comparing strategies is therefore: **sweep with this script + inspect runs in the MLflow UI**. There is no `--mode sweep` in the CLI by design — the CLI stays a single-config, single-run tool, and comparison belongs in MLflow.

---

5. MLflow tracking — `pads/tracking.py`

MLflow is an experiment tracker: a server that stores per-run parameters, metrics, tags and artifacts (models, plots) and provides a web UI to compare runs. In PADS, MLflow is **strictly opt-in**.

5.1 The opt-in contract

`pads/tracking.py` exposes `enabled()`, `check_connection()`, `run()`, `log_params`, `log_metric(s)`, `log_artifact`, `log_dict`, `file_sha256`. Every logging function checks `MLFLOW_TRACKING_URI` and **silently no-ops if the variable is not set**. The pipeline runs identically with or without MLflow installed.

This means MLflow is enabled per-shell, not per-machine: export `MLFLOW_TRACKING_URI` whenever you want runs recorded (laptop or server alike), and leave it unset for a quick local run that should not pollute the tracker.

Fail-soft connection check. When the variable *is* set, `cli.py` calls `check_connection()` once at startup: for an `http(s)` URI it probes the server's `/health` endpoint (file/sqlite stores are local, so they are skipped). If the server is unreachable, it prints a prominent warning and **disables tracking for the rest of the process** — the run still completes and writes models/artifacts to disk, you just don't get MLflow records. This avoids the old failure mode where a wrong URI (or `localhost` vs `127.0.0.1` on Windows) crashed the pipeline mid-run, after the data-prep work was already done.

5.2 What gets logged when enabled

Each pipeline step opens its own MLflow run (nested when called from `retrain_models()`):

Run	Logs
`retrain_models` (parent)	full `PADSConfig` as params; `dataset_sha256` tag
`retrain_mortality` (child)	last-epoch metrics (`mort/val_loss`, `mort/val_AUC`, …); the `.keras` artifact
`retrain_discharge` (child)	same, with `disch/` prefix
`calculate_metrics`	`test/mort_auc`, `test/mort_f1`, `test/disch_auc`, …; the ROC PNG
`inference`	`inf/<test_type>/mean_error`, `critical_error_rate`, …; `final_result.csv` + plots

The first 16 hex chars of the input dataset's SHA-256 are tagged on every run as `dataset_sha256`, so you can answer "which dataset trained this model?" from the MLflow UI.

5.3 Enabling tracking

```
# 1. Start an MLflow server (anywhere reachable from the training host).
#    Replace localhost with a remote hostname for a shared deployment.
mlflow server --host 0.0.0.0 --port 5000 \
              --backend-store-uri sqlite:///mlflow.db \
              --default-artifact-root /var/mlflow/artifacts
```

```
# 2. From the training host, point PADS at it
export MLFLOW_TRACKING_URI=http://localhost:5000
export PADS_MLFLOW_EXPERIMENT=pads_retraining        # optional
export PADS_RUN_TAGS="environment=local,operator=julen"   # optional

# 3. Run the pipeline as normal
uv run pads --data_filename synthetic_dataset.csv --mode all
```

Open `http://<host>:5000` to compare runs side by side.

5.4 Promoting a model to "Production"

```
mlflow models register -m runs:/<run_id>/mortality_model -n pads_mortality
mlflow models register -m runs:/<run_id>/discharge_model -n pads_discharge
# In the UI, transition the new version to "Production"
```

Loading the production model in code:

```
import mlflow.keras
mort_model  = mlflow.keras.load_model("models:/pads_mortality/Production")
disch_model = mlflow.keras.load_model("models:/pads_discharge/Production")
```

This second integration is **not wired into the pipeline yet** — it is a follow-up (Section 15).

5.5 Disabling locally

Just don't set `MLFLOW_TRACKING_URI` (leave it out of both the shell and `.env`). All tracking calls become no-ops; no artifacts are written, no metrics shipped. MLflow stays installed (it's a core dependency now), but it is never contacted.

5.6 `.env` file and Windows quirks

Repeating six exports before every run gets old. Drop them into a `.env` file at the repo root: `pads` **auto-loads** `<base_path>/.env` **at startup** (without overriding anything you already exported), so `uv run pads ...` picks them up with no extra step on any OS:

```
uv run pads --data_filename synthetic_dataset.csv --mode all
# [PADS] MLflow tracking ON: http://127.0.0.1:5000
```

For other tools that don't load it themselves (e.g. `mlflow ui`, a bare `pytest`), you can still pull `.env` into the current session manually:

```
. .\scripts\load_env.ps1              # Windows
. scripts/load_env.sh                 # Linux
```

A template lives at `.env.example` (committed); copy it to `.env` (gitignored) and fill in your credentials. Example content:

```
MLFLOW_TRACKING_URI=http://127.0.0.1:5000
MLFLOW_TRACKING_USERNAME=admin
MLFLOW_TRACKING_PASSWORD=...
PADS_MLFLOW_EXPERIMENT=pads_retraining
PADS_RUN_TAGS=environment=local,operator=julen
PYTHONIOENCODING=utf-8
PYTHONUTF8=1
TF_ENABLE_ONEDNN_OPTS=0
```

Two non-obvious workarounds bundled in there:

- **Use `127.0.0.1`, not `localhost`.** On Windows + Docker Desktop, `localhost` often resolves to `::1` (IPv6) while the container only binds IPv4, causing silent timeouts. `127.0.0.1` forces IPv4.

- `PYTHONIOENCODING=utf-8`**.** MLflow prints a unicode emoji ("View run ...") when closing a run; the Windows `cp1252` console codec cannot encode it and the pipeline crashes after the model has already been logged. Forcing UTF-8 stdout fixes it.

---

6. `pads/config.py` — pydantic

pydantic is a runtime data-validation library. `PADSConfig(BaseModel)` gives us three things for free:

1. **Type validation** — if you pass `epochs="ten"` you get a precise error at construction time instead of a crash 200 lines later. 2. **Constrained choices** — `retrain_type: RetrainType = "full"` where `RetrainType = Literal["full", "dense", "lstm", "scratch"]`. Trying to set it to anything else fails immediately. 3. **Derived fields** — `@model_validator(mode="after")` populates `inference_mort_model` from `retrain_type` if the user didn't set it explicitly.

Path helpers (`data_dir`, `model_dir`, `norm_dir`, `results_dir`) and `ensure_dirs()` keep the rest of the codebase free of `os.path.join` and `os.makedirs` calls.

The full field reference is in Section 11.

---

7. `pads/cli.py` — the CLI

```
python -m pads.cli --data_filename <file> --mode <step> [--base_path . --retrain_type full ...]
```

Highlights:

- `--mode all` runs every step in order (handy for fresh hospitals).
- `--retrain_type`, `--test_type`, `--seed`, `--epochs`, `--batch_size` are all CLI flags.
- Mode names are validated against an enum (`argparse choices`) — typos fail fast.
- `--data_filename` accepts both CSV and Parquet — the loader picks the reader from the extension.

After install, you can also drop the `python -m`:

```
pads --data_filename hospital_data.csv --mode all
```

---

8. Tests — `tests/`

The new package ships with **28 tests** (24 unit + 4 end-to-end), running in ~15 s.

```
pytest -m "not slow"      # fast feedback (skip E2E)
pytest                    # everything
pytest tests/test_windowing.py -v    # one file
```

File	What it covers
`test_schema.py`	dataset validation: valid → passes, missing column / non-numeric / missing file → raises
`test_preprocess.py`	outlier clipping, first-row imputation, NaN rules, full preprocess composition
`test_windowing.py`	rolling-window shape, time-reversal convention, discharge anchors, label helpers
`test_thresholds.py`	Youden / min-distance / PR threshold; regression test for the PR-index overflow

File	What it covers
`test_errors.py`	4-category mapping + severity 0..3
`test_pipeline_e2e.py`	smoke: full pipeline (1-epoch retrain) on `data/synthetic_dataset.csv`

`tests/conftest.py` defines two shared fixtures:

- `project_dir` — copies `data/synthetic_dataset.csv` into a clean tmp project layout.
- `project_dir_with_models` — same, plus copies the base `.keras` files from `models/`. Tests skip cleanly when the base models are not present, so a fresh clone still passes.

End-to-end tests are marked `slow` and `e2e` so CI can opt out of them when needed (e.g. `pytest -m "not slow"`).

---

9. Code-quality tooling

Optional static typing via `mypy` is configured but **not enforced automatically** (no pre-commit hook, no CI). You run it manually:

```
mypy pads                # type-check
```

The configuration lives in `pyproject.toml`. `ignore_missing_imports = true` accommodates the fact that TensorFlow's stubs are imperfect.

You can opt out without breaking the pipeline — see Section 14.

---

10. Two equivalent ways to run the pipeline

The same five steps are exposed via two interfaces. Pick the one that matches the situation.

10.1 Python API

```
from pads import PADSConfig, PADSPipeline

cfg = PADSConfig(
    base_path=".",
    retrain_type="full",   # or dense / lstm / scratch
    test_type="full",
    epochs=1000,
    batch_size=100,
    seed=42,
)
p = PADSPipeline(cfg)

p.prepare_data("hospital_data.csv")
p.retrain_models()
p.calculate_metrics()
errors = p.run_inference("hospital_data.csv", test_type="full")
```

Use this when calling the pipeline from a Jupyter notebook, another Python program, or unit tests.

10.2 CLI

```
python -m pads.cli --data_filename hospital_data.csv --mode all --base_path .

# or one step at a time
python -m pads.cli --data_filename hospital_data.csv --mode prepare_data
```

```
python -m pads.cli --data_filename hospital_data.csv --mode retrain_models \
                   --retrain_type full --epochs 1000
python -m pads.cli --data_filename hospital_data.csv --mode calculate_metrics
python -m pads.cli --data_filename hospital_data.csv --mode inference --test_type last_96h
```

Use this for one-off runs, debugging a single step, or production runs. To compare all four retrain strategies in one shot, wrap it with `scripts/sweep.ps1` / `scripts/sweep.sh` (Section 4.2).

---

11. Configuration reference

11.1 `PADSConfig` (pydantic)

Field	Default	Notes
`base_path`	`Path(".")`	project root
`retrain_mort_model`	`lstm_mortality_model.keras`	base model filename (inside `models/`)
`retrain_disch_model`	`lstm_disch_model.keras`	
`mort_normalizer`	`mortality_normalizer.pkl`	fitted on your data, inside `normalizers/`
`disch_normalizer`	`discharge_normalizer.pkl`	fitted on your data
`retrain_type`	`"full"`	`full / dense / lstm / scratch`
`test_type`	`"full"`	`full / last_48h / last_96h / first_48h`
`inference_mort_model`	derived from `retrain_type`	`RETRAINED_<retrain_type>_<retrain_mort_model>` if not overridden
`inference_disch_model`	derived from `retrain_type`	same pattern
`learning_rate_mort`	`1e-5`	
`learning_rate_disch`	`1e-5`	
`epochs`	`1000`	
`batch_size`	`100`	
`early_stopping_patience`	`50`	
`parallel`	`True`	currently unused (kept for API stability)
`n_jobs`	`20`	currently unused
`seed`	`42`	applied to Python, NumPy, TF

11.2 CLI flags

Every field above that you'd want to vary per run is also a CLI flag on `pads`:

```
pads --data_filename my_data.csv \   # filename inside data/ (CSV or Parquet)
     --mode all \                     # or a single step
     --retrain_type full \            # full | dense | lstm | scratch
     --test_type full \               # full | last_48h | last_96h | first_48h
     --epochs 1000 --batch_size 100 --seed 42 --base_path .
```

11.3 Environment variables

Variable	Effect
`MLFLOW_TRACKING_URI`	Enables MLflow tracking. If unset, all tracking is a no-op.
`PADS_MLFLOW_EXPERIMENT`	MLflow experiment name. Default: `pads`.
`PADS_RUN_TAGS`	`k1=v1,k2=v2` extra tags applied to every run.
`TF_ENABLE_ONEDNN_OPTS=0`	Disable oneDNN to silence float-rounding warnings.

---

12. The four pipeline steps in detail

12.1 Step 1 — `prepare_data(data_filename)`

Merges the former `generate_files` + `generate_retrain_data` steps: it loads the raw dataset once and produces every artifact the retraining needs, then records dataset provenance.

Reads: `data/<data_filename>` (full schema validation). **Writes:**

- `data/processed/medians_48h.json` — medians of `gcs_min`, `meanbp_min`, `bilirubin_max`, `platelet_min`, `creatinine_max` over `hr ≤ 48`.
- `data/processed/train_stays.txt`, `data/processed/test_stays.txt` — 80/20 split of stays with `los ≥ 48`.
- `normalizers/mortality_normalizer.pkl` — mortality MinMaxScaler, fit on your training data.
- `normalizers/discharge_normalizer.pkl` — discharge MinMaxScaler, fit on your training data.
- `data/processed/lstm_last_48h_{train,test}.pkl` — `dict[stay_id, (1, 48, F+1)]` with the most recent 48 h, time axis most-recent-first (mortality).
- `data/processed/icu_expire_flag_{train,test}.pkl` — `dict[stay_id, int]` (mortality).
- `data/processed/lstm_disch_3point_48h_{train,test}.pkl` — `dict[stay_id, (k, 48, F+1)]` with up to 5 anchor windows per stay (first, second, intermediate before/after, final), time axis oldest-first (discharge).
- `data/processed/outcome_disch_3point_48h_{train,test}.pkl` — `dict[stay_id, (k, 1)]` per-window discharge label.
- `data/processed/source_dataset.json` — provenance: `{dataset, dataset_sha256, created_at, n_train_stays, n_test_stays}` (the last two counted from `train_stays.txt` / `test_stays.txt`). Later steps that only load the `.pkl` files (`retrain_models`, `calculate_metrics`) read this to tag their MLflow run with the raw dataset and split sizes that produced their inputs.

The normalizers are fit only on **training stays** to avoid leakage. The committed `normalizers/mimic_iv_*.pkl` baselines (from the original MIMIC-IV fit) are left untouched — they are kept as a reference and are not used unless you point `mort_normalizer` / `disch_normalizer` at them.

12.2 Step 2 — `retrain_models()`

Loads the base models from `models/` and fine-tunes them. `retrain_type` controls the freezing strategy. Each model is `LSTM → Dense(50) → BatchNorm → … → Dense(2)`:

Value	LSTM	BatchNorm	Dense layers	Notes
`full`	trainable	trainable	trainable	fine-tune everything from the base model

Value	LSTM	BatchNorm	Dense layers	Notes
`dense`	**frozen**	trainable	trainable	keep the pretrained LSTM features, retrain the head
`lstm`	trainable	trainable	**frozen**	retrain the LSTM; BatchNorm re-fits so it doesn't diverge
`scratch`	random init	random init	random init	base model ignored, trained from scratch

`scratch` randomises **the whole model** (not just the LSTM). In `lstm`, only the `Dense` layers are frozen — BatchNorm stays trainable so it adapts to the retrained LSTM's outputs (freezing it caused NaN divergence on small datasets).

Writes:

- `models/RETRAINED_<retrain_type>_lstm_mortality_model.keras`
- `models/RETRAINED_<retrain_type>_lstm_disch_model.keras`
- `models/tensorflow_logs/RETRAIN_*/` — TensorBoard logs + best-checkpoint snapshots.

The parent MLflow run also logs `source_dataset.json` as an artifact (under `dataset/`) and tags every retrain run with `dataset` + `dataset_sha256` recovered from provenance.

12.3 Step 3 — `calculate_metrics()`

Predicts on the test set with the **retrained** models, picks an optimal threshold (default `min_distance`), and writes:

- `data/processed/model_parameters_test.json` — `{th_mort, th_disch, min_prob, max_prob}`.
- `results/<retrain_type>/images/roc_combined.png` — combined ROC + summary table.

12.4 Step 4 — `run_inference(data_filename, test_type=…)`

End-to-end inference plus error categorisation. `test_type` slices each stay to a chosen window:

Value	Window
`full`	all hours from `hr=47` onwards
`last_48h`	last 48 h
`last_96h`	last 96 h
`first_48h`	first 48 h

Writes:

- `results/<retrain_type>/final_result.csv` — per-prediction row with both probabilities, both categories, and severity 0..3.
- `results/<retrain_type>/model_parameters_inference.json`.
- `results/<retrain_type>/images/roc_combined_<test_type>.png`.
- `results/<retrain_type>/images/barplot_error.png` — error distribution.
- `results/<retrain_type>/images/heatmap_error.png` — confusion-matrix-style scatter.

---

13. Dataset schema and preprocessing

The input dataset (CSV or Parquet) must contain every column in `pads.data.schema.MANDATORY_COLUMNS`, all numeric:

```
stay_id, hr,
rate_epinephrine, rate_norepinephrine, rate_dopamine, rate_dobutamine,
meanbp_min, pao2fio2ratio_novent, pao2fio2ratio_vent, gcs_min,
bilirubin_max, creatinine_max, platelet_min,
admission_age, icu_expire_flag,
admission_type_Medical, admission_type_ScheduledSurgical,
admission_type_UnscheduledSurgical, charlson_comorbidity_index
```

`validate_dataset()` raises `DatasetValidationError` on any missing or non-numeric column.

The preprocessing pipeline (`pads.data.preprocess.preprocess`) does, in order:

1. Sort by `(stay_id, hr)`. 2. Clip each feature to its physiologically plausible range (`OUTLIER_CORRECTION`). 3. Impute the first hour of each stay with global medians for `gcs_min`, `meanbp_min`, `bilirubin_max`, `platelet_min`, `creatinine_max`. 4. Apply per-column NaN rules (`NAN_RULES`): `zero` for vasoactive rates and PaO■/FiO■ ratios; forward-fill within each stay for `gcs_min`, `meanbp_min`, `bilirubin_max`, `platelet_min`, `creatinine_max`.

Stays with `los < 48` are silently dropped from training — they appear in the input but contribute no windows.

---

14. What you could still trim

A frank pass through what's left, with a recommendation per item.

14.1 Worth keeping but with a small fix

- `PADSConfig.parallel` **and** `PADSConfig.n_jobs`**.** Currently unused (the trainer doesn't read them). Either wire them into a `joblib.Parallel` pre-processing pass, or delete them from the config to avoid future-self confusion.
- `inference_mort_model` **/** `inference_disch_model` **defaults.** They are set to the *base* model filenames in `config.py` but the `@model_validator` only fills them when they are `None`. Net effect: by default the pipeline runs inference with the base model, not the retrained one. If you always want the retrained version, change the defaults in `config.py` to `None` (so the validator picks the `RETRAINED_*` name). This is the single most likely cause of "why are my predictions identical to the base model?" surprises.

14.2 Safe to drop *if you don't use them*

- **MLflow integration.** If you decide tracking is overkill, delete `pads/tracking.py`, remove every `with tracking.run(...)` block from `pads/pipeline.py`, and drop `mlflow` from the core dependencies. The pipeline keeps working. Cost-of-keep is essentially zero, though — the wrapper is a no-op when `MLFLOW_TRACKING_URI` is unset, so the safe default is to leave it.
- `mypy`**.** If nobody on the team uses it, remove `[tool.mypy]` from `pyproject.toml` and drop it from the `dev` optional dependency group. `pytest` alone is enough to validate behaviour.

14.3 Do not remove

- `pads/` **package and** `tests/`**.** Obviously.

- `pyproject.toml` **itself.** The `[project]` table and `[tool.setuptools.packages.find]` are what make `pip install -e .` and `uv sync` work.
- `uv.lock`**.** Reproducibility hinges on it. If you drop it, anyone running `uv sync` may land on slightly different transitive versions than you did.

---

15. Known follow-ups

These are non-blocking — the pipeline is complete and tested as-is.

- **Calibration.** Today the "adjusted mortality %" in `final_result.csv` is a min-max normalisation, not a calibrated probability. Adding `CalibratedClassifierCV` or isotonic regression would make the % clinically interpretable.
- **Model registry.** When the MLflow server is up, wire `mlflow.register_model()` into `retrain_*` and load with `models:/pads_*/Production` in the inference path.
- **Pickles → parquet /** `.npz`**.** Pickles are fragile across NumPy versions. Migration is straightforward but breaks compatibility with existing `data/*.pkl`.
- **Docker.** A `Dockerfile` with pinned CUDA + TF for fully reproducible server runs.
- **CI.** GitHub Actions running `pytest -m "not slow"` on every push.

Appendix A — From scratch on a new hospital

```
# 1. Place data
Copy-Item C:\path\to\hospital_data.csv .\data\

# 2. Sanity-check on the laptop (use a low --epochs for a quick pass)
uv run pads --data_filename hospital_data.csv --mode all --epochs 5
Start-Process .\results\full\images\roc_combined.png   # results\<retrain_type>\...

# 3. Ship to the server for the real run
rsync -av data/hospital_data.csv server:~/pads/data/
ssh server
export MLFLOW_TRACKING_URI=http://mlflow:5000
cd pads
uv sync --all-extras
uv run pads --data_filename hospital_data.csv --mode all --epochs 1000

# 4. Inspect the runs at http://mlflow:5000
#    Promote the best one to Production from the UI
```

Appendix B — Architectural rationale (the "why")

- `pydantic` **for config, not** `dataclass`**.** Validation at boundary; derived fields (`inference_mort_model` from `retrain_type`); pretty CLI errors.
- **MLflow as opt-in no-op wrapper.** Same code path on laptop and server; MLflow is installed by default but only contacted when `MLFLOW_TRACKING_URI` is set, so a no-config run never touches it.
- **Manual CLI over a workflow engine.** The pipeline is a fixed linear chain of five steps run a handful of times, so a DAG engine (Snakemake/Make) added more concepts than it saved. Orchestration is a `for` loop in `scripts/sweep.*`; comparison lives in MLflow.

- **Two windowing conventions preserved.** Mortality stores most-recent-first, discharge stores oldest-first. This matches what the published base models expect — changing it would break compatibility with the released `.keras` files.
- **MinMaxScaler fit on `X[:, 0, :]` (first time slice only).** Mirrors the published pre-processing; documented in `pads/data/normalizer.py`.
- **Class weights: dual strategy.** Discharge uses `n / (2·n_positives)`; mortality uses sklearn `'balanced'` × `(0.8, 1.5)`. Matches the original models' training; centralised in `pads.training.trainer.class_weights`.
- `uv` **for env management.** Reproducible builds via `uv.lock`; single command for setup; same `pyproject.toml` works with plain `pip` for colleagues who prefer it.